

RUTA DE APRENDIZAJE · PROGRAMACIÓN EN JAVA

Java 2026 — De cero a experto

CAPÍTULO

02

Operadores y control de flujo

Enseña a tu programa a calcular, comparar, decidir y repetir — con la sintaxis moderna de Java.

EN ESTE CAPÍTULO APRENDERÁS

Operadores aritméticos, lógicos y bitwise · Ternario y precedencia · if / else · switch clásico y moderno · Pattern matching (Java 21) · while, do-while, for y for-each · break, continue y return

5 h

TIEMPO ESTIMADO

4

SECCIONES

Inicial+

NIVEL

6

RETOS

Libro-tutorial paso a paso · Edición a color 2026

Mapa del capítulo

Cuatro paradas que convierten tus programas lineales en programas que piensan: primero las herramientas (operadores), luego las decisiones, después la repetición y, por último, el control fino del flujo.

Nº	Sección	Al terminarla podrás...
1	Operadores: el motor que calcula y decide	Usar las 6 familias de operadores y domar la precedencia.
2	Tomar decisiones: control condicional	Elegir caminos con if/else, switch moderno y pattern matching.
3	Repetir tareas: los bucles	Escoger el bucle correcto: for, while, do-while o for-each.
4	Controlar el flujo: break, continue y return	Salir, saltar o abandonar sin perderte en el camino.

En este capítulo aprenderás a...

- Usar todos los operadores de Java: aritméticos, de asignación, relacionales, lógicos, bitwise y el ternario.
- Entender la precedencia y por qué los paréntesis son tus mejores amigos.
- Tomar decisiones con if, if-else y switch, incluido el switch moderno y el pattern matching de Java 21.
- Repetir tareas con while, do-while, for y for-each, y saber cuándo usar cada uno.
- Controlar el flujo con break, continue y return sin perderte.



Java moderno (2026)

Java avanza con versiones LTS frecuentes (17, 21, 25...). Este capítulo te enseña primero la forma clásica — la que verás en código antiguo— y luego la forma moderna y recomendada hoy. Cuando ambas existan, prioriza la moderna: es más corta, más segura y más fácil de leer.

1

Operadores: el motor que calcula y decide

Seis familias de herramientas para transformar y comparar datos



¿Por qué importa?

Todo programa útil calcula algo: un precio con descuento, una edad, si una contraseña es válida. Los operadores son el vocabulario de esos cálculos; sin ellos, tus variables solo guardan datos sin hacer nada con ellos.

Un operador es un símbolo que realiza una acción sobre uno o más valores, llamados operandos. Si las variables son las cajas donde guardas datos, los operadores son las herramientas que los transforman y comparan. Vamos por familias.

1.1 Operadores aritméticos

Hacen las cuentas de toda la vida. El único “raro” es el módulo (%), que devuelve el residuo de una división.

Operador	Nombre	Ejemplo	Resultado
+	Suma	7 + 3	10
-	Resta	7 - 3	4
*	Multiplicación	7 * 3	21
/	División	7 / 3	2 (¡entera!)
%	Módulo (residuo)	7 % 3	1

```

Aritmetica.java

1 int a = 7, b = 3;
2 System.out.println(a + b); // 10
3 System.out.println(a / b); // 2 -> división ENTERA, se pierde el decimal
4 System.out.println(a % b); // 1 -> el residuo de 7 entre 3
5
6 double x = 7.0;
7 System.out.println(x / b); // 2.3333... -> ahora sí hay decimales

```

⚠ Error común: la división entera

Si divides dos enteros, Java tira la parte decimal: 7 / 3 da 2, no 2.33. Para obtener decimales, al menos uno de los operandos debe ser decimal: usa 7.0 / 3 o convierte con (double) a / b.

⚡ El módulo es más útil de lo que parece

¿Número par o impar? → $n \% 2 == 0$ significa par.

¿Último dígito de un número? → $n \% 10$.

¿Avanzar en círculo (relojes, turnos)? → $(i + 1) \% \text{total}$.

Incremento y decremento: ++ y --

Suman o restan 1. La posición importa: si va antes (++i, pre-incremento) cambia el valor ANTES de usarlo; si va después (i++, post-incremento) lo usa y LUEGO lo cambia.

```
Incremento.java
1 int i = 5;
2 System.out.println(i++); // imprime 5, luego i pasa a 6
3 System.out.println(++i); // i pasa a 7, luego imprime 7
4
5 int contador = 0;
6 contador++; // forma habitual de "sumar uno"
```

1.2 Operadores de asignación

El = guarda un valor en una variable. Los compuestos (+=, -=, etc.) son atajos: x += 3 es exactamente lo mismo que x = x + 3.

Atajo	Equivale a	Ejemplo (x=10)	Resultado
+=	x = x + y	x += 5	15
-=	x = x - y	x -= 5	5
*=	x = x * y	x *= 2	20
/=	x = x / y	x /= 2	5
%=	x = x % y	x %= 3	1

```
Asignacion.java
1 int saldo = 100;
2 saldo += 50; // ahora 150 (depósito)
3 saldo -= 30; // ahora 120 (retiro)
4 saldo *= 2; // ahora 240
5 System.out.println(saldo); // 240
```

1.3 Operadores relacionales (de comparación)

Comparan dos valores y siempre devuelven un boolean (true o false). Son el corazón de cualquier decisión.

Operador	Significa	Ejemplo	Resultado
==	¿son iguales?	5 == 5	true
!=	¿son distintos?	5 != 3	true

Operador	Significa	Ejemplo	Resultado
>	¿mayor que?	5 > 3	true
<	¿menor que?	5 < 3	false
>=	¿mayor o igual?	5 >= 5	true
<=	¿menor o igual?	3 <= 5	true

⚠️ == NO sirve para comparar textos

Con números, == funciona perfecto. Pero con objetos como String, == compara si son el MISMO objeto en memoria, no si tienen el mismo contenido. Para comparar el texto usa .equals():

"hola".equals(otro) → correcto. texto == otro → trampa.

1.4 Operadores lógicos

Combinan condiciones booleanas. Con ellos expresas ideas como “si tiene entrada Y es mayor de edad”.

Tablas de verdad de los operadores lógicos

&& (Y)			(O)			! (NO)	
A	B	=	A	B	=	A	!A
V	V	V	V	V	V	V	F
V	F	F	V	F	V	F	V
F	V	F	F	V	V		
F	F	F	F	F	F		

V solo si AMBOS son V

V si AL MENOS UNO es V

Invierte el valor

Figura 2.1 — Tablas de verdad de &&, || y !.

Logicos.java

```

1 boolean tieneEntrada = true;
2 int edad = 20;
3
4 boolean puedeEntrar = tieneEntrada && edad >= 18; // Y: ambas verdaderas
5 boolean hayDescuento = edad < 12 || edad > 65; // O: alguna verdadera
6 boolean menorDeEdad = !(edad >= 18); // NO: invierte
7
8 System.out.println(puedeEntrar); // true

```

⚡ Cortocircuito: && y || son perezosos

Java evalúa lo justo y necesario. En `A && B`, si `A` ya es `false`, ni mira `B` (el resultado ya es `false`). En `A || B`, si `A` ya es `true`, ni mira `B`.

Esto te protege: en `obj != null && obj.activo` el segundo trozo solo se evalúa si `obj` no es `null`, evitando un error.

1.5 Operadores bit a bit (bitwise)

Trabajan sobre los bits individuales de los números enteros. Son menos frecuentes en el día a día, pero clave en rendimiento, banderas, criptografía y bajo nivel.

Operador	Nombre	Qué hace
<code>&</code>	AND	1 solo si ambos bits son 1
<code> </code>	OR	1 si al menos un bit es 1
<code>^</code>	XOR	1 si los bits son distintos
<code>~</code>	NOT	invierte todos los bits
<code><<</code>	Shift izquierda	desplaza bits a la izquierda (×2 por posición)
<code>>></code>	Shift derecha	desplaza bits a la derecha (÷2 por posición)
<code>>>></code>	Shift derecha sin signo	rellena con ceros por la izquierda

Bitwise.java

```

1  int a = 6;    // en binario: 0110
2  int b = 3;    // en binario: 0011
3
4  System.out.println(a & b);    // 2  -> 0010
5  System.out.println(a | b);    // 7  -> 0111
6  System.out.println(a ^ b);    // 5  -> 0101
7  System.out.println(a << 1);   // 12 -> 1100 (multiplica por 2)
8  System.out.println(a >> 1);   // 3  -> 0011 (divide entre 2)

```

🔑 No confundas & con &&

`&&` y `||` son lógicos (trabajan con booleanos y hacen cortocircuito). `&` y `|` son bit a bit (trabajan con los bits de enteros). Para condiciones normales usa siempre la versión doble: `&&` y `||`.

1.6 Operador ternario (? :)

Es un if-else en miniatura que devuelve un valor. Se lee: condición ? valorSiVerdadero : valorSiFalso.

```

Ternario.java

1  int edad = 20;
2
3  // Versión larga con if-else:
4  String tipo;
5  if (edad >= 18) { tipo = "adulto"; } else { tipo = "menor"; }
6
7  // Versión corta con ternario (¡misma lógica!):
8  String tipo2 = edad >= 18 ? "adulto" : "menor";
9
10 int mayor = (a > b) ? a : b; // devuelve el mayor de dos números
    
```

⚡ Buena práctica 2026

Usa el ternario para asignaciones cortas y claras. Si necesitas anidar varios ternarios o el resultado se vuelve difícil de leer, prefiere un if-else o un switch: la legibilidad gana siempre.

1.7 Precedencia de operadores

Cuando hay varios operadores en una expresión, Java decide cuál aplica primero según su precedencia (igual que en matemáticas, primero multiplicar y luego sumar).

Precedencia de operadores

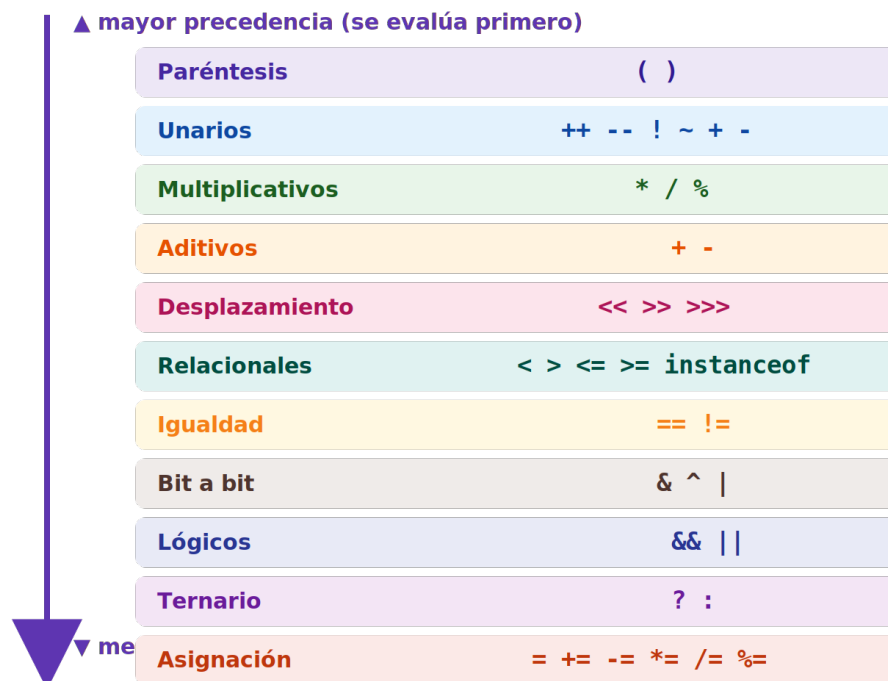


Figura 2.2 — De mayor (arriba) a menor (abajo) precedencia.

🔑 La regla de oro de la precedencia

No memorices toda la tabla. En su lugar, usa paréntesis para dejar tus intenciones claras. Compara:

$2 + 3 * 4$ da **14**, pero $(2 + 3) * 4$ da **20**.

Un paréntesis de más nunca está de más: hace el código legible y evita errores.

✅ Punto de control

- ☐ Puedo predecir el resultado de $7 / 2$, $7 \% 2$ y $7.0 / 2$ sin ejecutarlos.
- ☐ Sé por qué `"a".equals(b)` es correcto y `a == b` es una trampa con textos.
- ☐ Distingo `&&` (lógico) de `&` (bitwise) y sé cuál usar en un `if`.

2

Tomar decisiones: control condicional

Del if clásico al pattern matching de Java 21

💡 ¿Por qué importa?

Hasta ahora el código se ejecutaba de arriba abajo, línea por línea. Las estructuras condicionales permiten que el programa elija qué camino seguir según se cumplan o no ciertas condiciones: aprobar o reprobar, cobrar o rechazar, entrar o quedarse fuera.

¿Cómo decide un if-else?

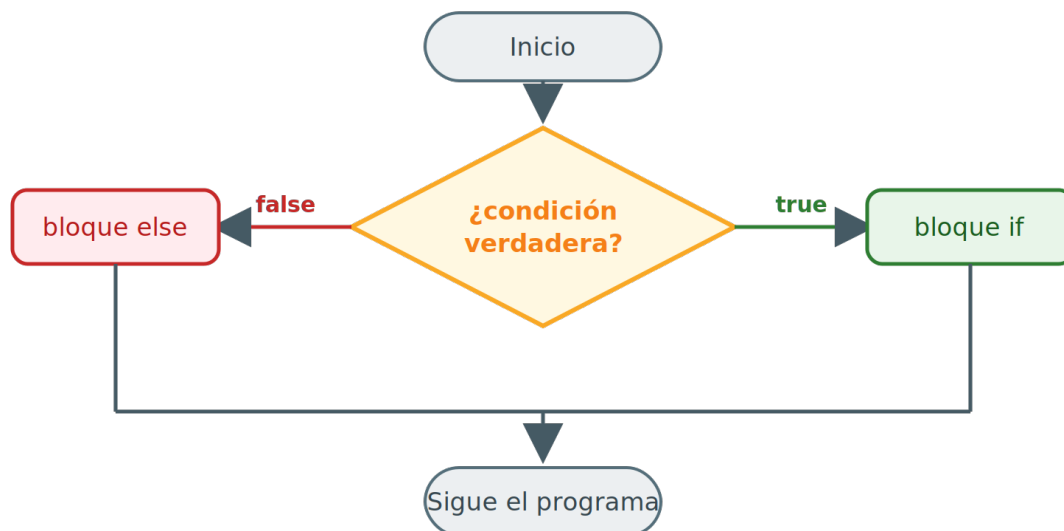


Figura 2.3 — Cómo decide un if-else qué bloque ejecutar.

2.1 La sentencia if

Ejecuta un bloque solo si la condición es verdadera. Si es falsa, simplemente lo salta.

```
● ● ● If.java

1 int temperatura = 38;
2
3 if (temperatura > 37.5) {
4     System.out.println("Tienes fiebre.");
5 }
```

2.2 if-else: el camino alternativo

Añade un plan B: qué hacer cuando la condición NO se cumple.

```
● ● ● IfElse.java

1 int nota = 6;
2
3 if (nota >= 6) {
4     System.out.println("Aprobado ✓");
5 } else {
6     System.out.println("Reprobado ✗");
7 }
```

2.3 Cadena if / else if / else

Para más de dos caminos, encadena condiciones. Java evalúa de arriba abajo y ejecuta el PRIMER bloque cuya condición sea verdadera.

```
● ● ● Cadena.java

1 int nota = 85;
2
3 if (nota >= 90) {
4     System.out.println("Sobresaliente");
5 } else if (nota >= 80) {
6     System.out.println("Notable");
7 } else if (nota >= 60) {
8     System.out.println("Aprobado");
9 } else {
10    System.out.println("Reprobado");
11 }
```

⚠ Error común: usar = en vez de ==

if (x = 5) es asignación, no comparación. Quieres if (x == 5). Con números, Java te avisará del error; pero con booleanos puede colarse silenciosamente. Revisa siempre que uses == para comparar.

2.4 switch clásico

Cuando comparas UNA variable contra muchos valores posibles, el switch es más limpio que una cadena enorme de if. La forma tradicional usa case, break y, opcionalmente, default.

```
SwitchClasico.java

1 int dia = 3;
2 String nombre;
3
4 switch (dia) {
5     case 1: nombre = "Lunes"; break;
6     case 2: nombre = "Martes"; break;
7     case 3: nombre = "Miércoles"; break;
8     default: nombre = "Día inválido";
9 }
10 System.out.println(nombre); // Miércoles
```

⚠ El temido “fall-through”

En el switch clásico, si olvidas el break, la ejecución “se cae” al siguiente case y sigue ejecutando los demás. Es una de las fuentes de bugs más clásicas de Java. Por eso existe la sintaxis moderna que verás ahora.

2.5 switch moderno: expresión con flecha (Java 14+)

Desde Java 14, el switch puede usar la flecha -> y comportarse como una expresión que DEVUELVE un valor. No hay fall-through, no necesitas break, y puedes agrupar varios casos con comas.

```
SwitchModerno.java

1 int dia = 3;
2
3 String nombre = switch (dia) {
4     case 1 -> "Lunes";
5     case 2 -> "Martes";
6     case 3 -> "Miércoles";
7     case 6, 7 -> "Fin de semana"; // varios casos agrupados
8     default -> "Día inválido";
9 };
10 System.out.println(nombre); // Miércoles
```

Si un caso necesita varias líneas, usa llaves y la palabra yield para devolver el valor:

```
SwitchYield.java

1 int puntos = 75;
2
3 String rango = switch (puntos / 10) {
4     case 10, 9 -> "Oro";
5     case 8, 7 -> {
6         System.out.println("Buen trabajo");
7         yield "Plata"; // yield devuelve el valor del bloque
8     }
9     default -> "Bronce";
10 };
```

⚡ Buena práctica 2026

Prefiere SIEMPRE el switch con flecha (->) sobre el clásico con break. Es más corto, no sufre fall-through accidental y, al ser una expresión, suele dejar el código más limpio. El switch clásico solo lo verás en código antiguo.

2.6 Pattern matching: el superpoder moderno (Java 21)

El pattern matching (coincidencia de patrones) permite preguntar “¿de qué tipo es esto?” y, a la vez, usarlo ya convertido, sin casts manuales. Empezó con instanceof y culminó en el switch de Java 21.

instanceof con patrón

Antes había que comprobar el tipo y luego convertir a mano. Ahora se hace en un solo paso, declarando la variable ya con el tipo correcto:

```
InstanceofPatron.java

1 Object obj = "Hola mundo";
2
3 // Forma antigua:
4 if (obj instanceof String) {
5     String s = (String) obj; // cast manual
6     System.out.println(s.length());
7 }
8
9 // Forma moderna (Java 16+): el patrón crea la variable s ya convertida
10 if (obj instanceof String s) {
11     System.out.println(s.length()); // s ya es un String
12 }
```

switch con patrones de tipo y guardas

El switch moderno puede ramificar según el TIPO del valor, manejar el caso null de forma explícita y añadir condiciones extra con la palabra when (las llamadas “guardas”):

```

SwitchPatrones.java

1 static String describir(Object valor) {
2     return switch (valor) {
3         case null -> "No hay nada";
4         case Integer i when i < 0 -> "Entero negativo: " + i;
5         case Integer i -> "Entero: " + i;
6         case String s -> "Texto de " + s.length() + " letras";
7         case Double d -> "Decimal: " + d;
8         default -> "Tipo desconocido";
9     };
10 }

```

Y con records, el pattern matching puede incluso desarmar el objeto y sacar sus campos directamente (record patterns):

```

RecordPattern.java

1 record Punto(int x, int y) {}
2
3 static String ubicar(Object o) {
4     return switch (o) {
5         case Punto(int x, int y) when x == 0 && y == 0 -> "En el origen";
6         case Punto(int x, int y) -> "Punto en (" + x + ", " + y + ")";
7         default -> "No es un punto";
8     };
9 }

```



¿Por qué importa esto en 2026?

El pattern matching hace que el código sea más seguro y expresivo: menos casts, menos errores, y el compilador puede avisarte si olvidas cubrir algún caso. Es uno de los grandes cambios del Java moderno, así que conviene acostumbrarte desde ya.



Punto de control

- ☐ Sé encadenar if / else if / else y explicar en qué orden se evalúan.
- ☐ Puedo reescribir un switch clásico como switch con flecha (->).
- ☐ Entiendo qué hace case String s -> ... en un switch de Java 21.



Glosario exprés

Fall-through: caída de la ejecución al siguiente case cuando falta un break en el switch clásico.

Guarda (when): condición extra que debe cumplirse para que un case con patrón aplique.

Cast: conversión manual de un tipo a otro, p. ej. (String) obj. El pattern matching lo evita.

3

Repetir tareas: los bucles

while, do-while, for y for-each: cada uno brilla en su momento



¿Por qué importa?

Un bucle (o ciclo) repite un bloque de código mientras se cumpla una condición. Es lo que evita que copies y pegues la misma línea mil veces: procesar 10 000 ventas, validar un menú o recorrer una lista es siempre trabajo de bucles.

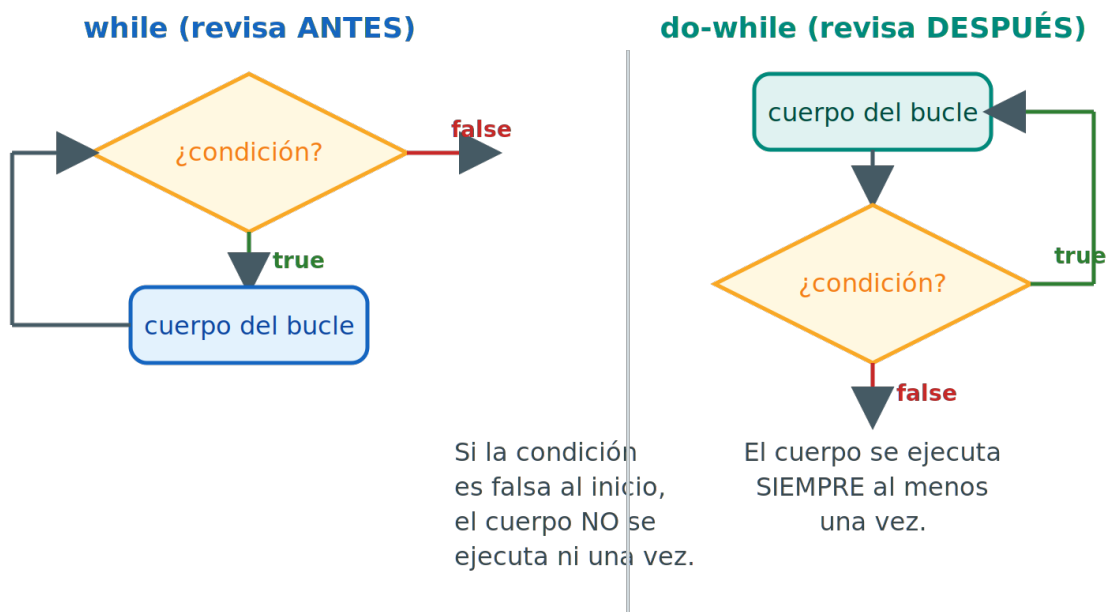


Figura 2.4 — while revisa la condición antes; do-while, después.

3.1 while: “mientras se cumpla...”

Revisa la condición ANTES de cada vuelta. Si es falsa desde el inicio, el cuerpo no se ejecuta ni una vez.

While.java

```

1  int contador = 1;
2
3  while (contador <= 5) {
4      System.out.println("Vuelta " + contador);
5      contador++;    // ¡imprescindible para que termine!
6  }
    
```

⚠ Cuidado con el bucle infinito

Si la condición nunca se vuelve falsa (por ejemplo, olvidas el contador++), el bucle se repite para siempre y el programa se congela. Asegúrate de que algo dentro del bucle acerque la condición a su fin.

3.2 do-while: ejecuta primero, pregunta después

Garantiza que el cuerpo se ejecute AL MENOS una vez, porque la condición se revisa al final. Perfecto para menús y validación de entrada.

```

DoWhile.java

1  int opcion;
2
3  do {
4      System.out.println("1) Jugar   2) Salir");
5      opcion = sc.nextInt();
6  } while (opcion != 2); // repite hasta que elija salir
    
```

3.3 for: el bucle con contador

Ideal cuando sabes cuántas veces repetir. Reúne en una sola línea las tres partes del control: inicio, condición y avance.

```

For.java

1  // inicio ; condición ; avance
2  for (int i = 0; i < 5; i++) {
3      System.out.println("i vale " + i);
4  }
5  // Imprime i = 0, 1, 2, 3 y 4
    
```

Las tres partes del for, paso a paso:

Parte	Cuándo se ejecuta	En el ejemplo
Inicio	Una sola vez, al principio.	<code>int i = 0</code>
Condición	Antes de cada vuelta. Si es false, termina.	<code>i < 5</code>
Avance	Al final de cada vuelta.	<code>i++</code>

3.4 for-each: recorrer colecciones sin contador

Cuando solo quieres recorrer todos los elementos de un arreglo o lista (sin importar el índice), el for-each es más limpio y seguro. Se lee: “para cada elemento de...”.

```

ForEach.java

1 String[] frutas = { "manzana", "banana", "cereza" };
2
3 for (String fruta : frutas) {
4     System.out.println("Me gusta la " + fruta);
5 }
    
```

Consola / Terminal

```

Me gusta la manzana
Me gusta la banana
Me gusta la cereza
    
```

¿Cuál bucle usar? Guía rápida

Situación	Bucle recomendado
Sé exactamente cuántas veces repetir	for
Repetir mientras algo sea cierto (veces desconocidas)	while
Necesito ejecutar al menos una vez (menús, validación)	do-while
Recorrer todos los elementos de un arreglo o lista	for-each

Manos a la obra

Escribe un programa que imprima la cuenta regresiva 10, 9, 8... 1, ¡Despegue! tres veces: primero con for, luego con while y por último con do-while. Verás en la práctica que los tres pueden hacer lo mismo, pero uno se siente más natural.

Punto de control

- ☐ Sé qué imprime for (int i = 0; i < 5; i++) sin ejecutarlo.
- ☐ Puedo explicar la diferencia entre while y do-while con un ejemplo de menú.
- ☐ Reconozco cuándo un bucle será infinito y sé cómo evitarlo.

4

Controlar el flujo: break, continue y return

Salir, saltar o abandonar — con criterio

Dentro de los bucles puedes cambiar el flujo normal con tres palabras clave. Úsalas con criterio: bien empleadas aclaran el código; abusar de ellas lo enreda.

4.1 break: salir del bucle de inmediato

Detiene el bucle por completo y continúa con el código que viene después de él.

```
Break.java

1 for (int i = 1; i <= 10; i++) {
2     if (i == 5) {
3         break;    // al llegar a 5, abandona el bucle
4     }
5     System.out.println(i);    // imprime 1, 2, 3, 4
6 }
```

4.2 continue: saltar a la siguiente vuelta

Omite el resto del cuerpo en la vuelta actual y pasa directamente a la siguiente iteración. El bucle NO termina.

```
Continue.java

1 for (int i = 1; i <= 6; i++) {
2     if (i % 2 != 0) {
3         continue;    // si es impar, salta a la siguiente vuelta
4     }
5     System.out.println(i);    // imprime solo pares: 2, 4, 6
6 }
```

4.3 return: salir del método entero

A diferencia de break (que solo sale del bucle), return abandona el método completo y, si corresponde, devuelve un valor. Cualquier código posterior dentro del método no se ejecuta.

```
Return.java

1 static boolean contiene(int[] numeros, int objetivo) {
2     for (int n : numeros) {
3         if (n == objetivo) {
4             return true;    // lo encontró: sale YA del método
5         }
6     }
7     return false;    // recorrió todo y no estaba
8 }
```

break vs continue vs return

break → sale del bucle, sigue en el método.

continue → salta a la siguiente vuelta del bucle.

return → abandona el método entero (y opcionalmente devuelve un valor).

4.4 Etiquetas: break y continue en bucles anidados

Cuando tienes un bucle dentro de otro, break y continue afectan solo al más interno. Con una etiqueta puedes decirle a Java a qué bucle te refieres. Es poco frecuente, pero útil para salir de todo de golpe.

Etiquetas.java

```
1 buscar: // etiqueta del bucle externo
2 for (int fila = 0; fila < 3; fila++) {
3     for (int col = 0; col < 3; col++) {
4         if (tablero[fila][col] == 'X') {
5             System.out.println("Encontrado en " + fila + "," + col);
6             break buscar; // sale de AMBOS bucles a la vez
7         }
8     }
9 }
```

⚡ Buena práctica 2026

Las etiquetas se usan muy poco: a menudo, si necesitas saltar de varios bucles, conviene extraer esa lógica a un método y usar return, que resulta más claro. Resérvalas para casos genuinamente complejos.

🚩 RETOS DEL CAPÍTULO

Resuelve cada reto escribiendo y ejecutando el código. Intenta primero sin mirar la pista. Los últimos son pequeños proyectos que combinan todo lo aprendido.

🏆 Reto 1 · Par o impar ★★★

Lee un número entero y muestra si es par o impar usando el operador módulo y un if-else.

💡 **Pista:** $n \% 2 == 0$ significa par.

🏆 Reto 2 · Mayor de tres ★★★

Lee tres números y muestra el mayor. Hazlo dos veces: una con if-else y otra con el operador ternario.

💡 **Pista:** Combina dos ternarios o compara por pasos.

🏆 Reto 3 · Tabla de multiplicar ★★★

Pide un número del 1 al 10 y muestra su tabla de multiplicar completa usando un bucle for.

💡 **Pista:** *for (int i = 1; i <= 10; i++) e imprime $n * i$.*

🏆 Reto 4 · FizzBuzz ★★★

Recorre del 1 al 100. Imprime “Fizz” si el número es múltiplo de 3, “Buzz” si lo es de 5, “FizzBuzz” si lo es de ambos, y el número en cualquier otro caso.

💡 **Pista:** *Comprueba primero el caso de ambos (% 15) o concatena las cadenas.*

🏆 Reto 5 · Clasificador moderno ★★★

Escribe un método que reciba un Object y, usando un switch con pattern matching, devuelva una descripción según sea Integer, String, Double o null.

💡 **Pista:** *Inspírate en el ejemplo de la sección 2.6 y usa case con patrones de tipo.*

🏆 Reto 6 · Menú de cajero (proyecto) ★★★

Crea un menú con do-while que se repita hasta elegir “Salir”. Las opciones: consultar saldo, depositar y retirar. Valida que no se pueda retirar más del saldo disponible y usa operadores de asignación compuesta.

💡 **Pista:** *do { mostrar menú; leer opción; switch(opcion) {...} } while (opcion != salir). Usa saldo += y saldo -=.*

✓ RESUMEN DEL CAPÍTULO

📖 Lo esencial que te llevas

- Los operadores se agrupan en aritméticos, de asignación, relacionales, lógicos, bit a bit y el ternario.
- La división entre enteros descarta los decimales; el módulo (%) da el residuo y es muy versátil.
- && y || hacen cortocircuito; usa .equals() (no ==) para comparar textos.
- Ante dudas de precedencia, usa paréntesis para mayor claridad.
- if / else if / else y switch eligen caminos; prefiere el switch moderno con -> y el pattern matching de Java 21.
- for, while, do-while y for-each repiten tareas; cada uno brilla en una situación distinta.
- break sale del bucle, continue salta a la siguiente vuelta y return abandona el método.



Chuleta de decisiones y bucles

Quiero...	Uso...
Elegir entre 2 caminos	<code>if / else</code>
Elegir entre muchos valores de una variable	<code>switch (dia) { case 1 -> ... }</code>
Decidir según el TIPO de un objeto	<code>switch con pattern matching (Java 21)</code>
Asignar un valor corto según una condición	<code>condición ? valorA : valorB</code>
Repetir un número conocido de veces	<code>for (int i = 0; i < n; i++)</code>
Repetir hasta que algo cambie	<code>while / do-while</code>
Recorrer una colección completa	<code>for (Tipo elem : coleccion)</code>



Autoevaluación: ¿puedo decir que sí?

Marca cada afirmación solo si puedes demostrarla escribiendo código o explicándosela a alguien más. Si alguna queda sin marcar, vuelve a la sección indicada antes de pasar al Capítulo 3.

Afirmación	Repasa en...
☐ Puedo predecir el resultado de expresiones con /, % y paréntesis.	Secciones 1.1, 1.7
☐ Comparo textos con <code>.equals()</code> y sé por qué <code>==</code> falla.	Sección 1.3
☐ Escribí un switch moderno con <code>-></code> y agrupé casos con comas.	Sección 2.5
☐ Usé <code>instanceof</code> con patrón o un switch con patrones de tipo.	Sección 2.6
☐ Elijo el bucle correcto (<code>for</code> , <code>while</code> , <code>do-while</code> , <code>for-each</code>) sin dudar.	Sección 3
☐ Sé cuándo usar <code>break</code> , <code>continue</code> y <code>return</code> dentro de un bucle.	Sección 4
☐ Resolví al menos 4 de los 6 retos, incluido FizzBuzz.	Retos